

Intro to Programming in R: Fundamentals (Code Chunks)

Selene R. Schmittling

Variables in R

Will it Variable?

```
# Set some variables:  
wolf.name <- "Smiley"  
wolf.age <- 5  
  
# See the variable value  
print(wolf.name)
```

```
[1] "Smiley"
```

```
print(wolf.age)
```

```
[1] 5
```

Data Types

Numeric

```
# Create some variables and set them to numeric values
numWolves <- 5
pctWolves <- .20

# Add a decimal place to numWolves
numWolves <- 5.0

print(numWolves)
```

```
[1] 5
```

```
print(typeof(numWolves))
```

```
[1] "double"
```

```
print(as.double(numWolves))
```

```
[1] 5
```

Text

```
# Some text examples

wolf.name <- "Scruffy"
wolf.initial <- "D"
wolf.FavoriteArtist <- "Prince"

print(wolf.name)
```

```
[1] "Scruffy"
```

```
print(wolf.initial)
```

```
[1] "D"
```

```
print(wolf.FavoriteArtist)
```

```
[1] "Prince"
```

```
print(typeof(wolf.name))
```

```
[1] "character"
```

Dates & Times

```
# define a variable and set it to a text date  
dateStr <- "07/15/2025"  
  
print(dateStr)
```

```
[1] "07/15/2025"
```

```
# addDays <- dateStr + 4
```

```
# Let's try some dates  
  
# create text date  
bday.str <- "8/18/2019"  
  
# convert using as.Date()  
bday.1 <- as.Date(bday.str, format = "%m/%d/%Y")  
print(bday.1)
```

```
[1] "2019-08-18"
```

```
# convert using as.POSIXct()  
bday.2 <- as.POSIXct("11-01-2015", format="%m-%d-%Y")  
print(bday.2)
```

```
[1] "2015-11-01 EDT"
```

```
# Define a date using long names
bday.long <- format(bday.1, "%A, %B %d, %Y")
print(bday.long)
```

```
[1] "Sunday, August 18, 2019"
```

```
# Define an anniversary date using bday.1
anniversary <- bday.1 + 63

print(anniversary)
```

```
[1] "2019-10-20"
```

Logical

```
# Example of boolean variables based on picture. Define a values for isAlive and isWolf.1
isAlive = FALSE
isWolf.1 = TRUE

print(isWolf.1)
```

```
[1] TRUE
```

```
print(isAlive)
```

```
[1] FALSE
```

```
# What do you think? Uncomment the two lines below and set them to the value you think they s

#isMajestic =
#isWolf.2 =

# print(isWolf.2)
# print(isMajestic)
```

Data Structures

Vectors (1-D Arrays)

Vectors store data in one dimension. They allow only one data type.

How to Create Vectors

```
# EXAMPLES OF HOW TO CREATE VECTORS
# (Single Data Type)

# Numeric
# Since R sees integers and decimal data as double, vectors can contain both
num.vec <- c(10, 4.5, 7, 12.2, 3.0)
print(num.vec)
```

```
[1] 10.0  4.5  7.0 12.2  3.0
```

```
print(typeof(num.vec))
```

```
[1] "double"
```

```
# Examples using other data types
txt.vec <- c("Talley", "SAS Hall", "Hunt Library")
log.vec <- c(TRUE, FALSE, TRUE, TRUE)
```

```
# Let's store some info about a pet cat named Fluffy: name, weight, isFemale, birthdate and age
mixed.vec <- c("Fluffy", 12.0, TRUE, as.Date("9/1/2024", format="%m/%d/%Y"), 2) # so far so good
print(mixed.vec) # oops...
```

```
[1] "Fluffy" "12"      "TRUE"    "19967"   "2"
```

Matrices (or n-D Arrays)

```
# generate 35 random values between 0 and 50
data <- sample(0:50, size = 35, replace = TRUE)
print(data)
```

```
[1] 24 28 28 41 36 39 47 46 31  3 11 33 32 22 33  5 29 14 34 20  5  4 34 45 21
[26] 35  5  8 27  1 41  1 43 19 50
```

```
# make a 2-D matrix out of data
matrix.2D <- matrix(data, nrow = 5, ncol = 7, byrow = FALSE)
print(matrix.2D)
```

```
      [,1] [,2] [,3] [,4] [,5] [,6] [,7]
[1,]    24    39    11     5     5    35    41
[2,]    28    47    33    29     4     5     1
[3,]    28    46    32    14    34     8    43
[4,]    41    31    22    34    45    27    19
[5,]    36     3    33    20    21     1    50
```

```
# generate 75 random values between 0 and 50
data <- sample(0:50, size = 75, replace = TRUE)

# make a 3-D matrix from data
matrix.3D <- array(data,
                    dim = c(5, 5, 3))
print(matrix.3D)
```

```
, , 1
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,]    42    50    18    22    47
[2,]    13     7     9    30    37
[3,]    46    37    41    29    33
[4,]     6    16    28     2    43
[5,]    22     3     8    22    41
```

```
, , 2
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	33	10	24	32	43
[2,]	17	36	14	13	19
[3,]	18	16	27	4	10
[4,]	44	46	11	11	50
[5,]	1	15	45	22	13

, , 3

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	4	43	30	28	40
[2,]	20	41	12	16	13
[3,]	2	31	23	44	1
[4,]	3	16	5	33	43
[5,]	48	41	18	45	25

Dataframes (Tabular Data)

```
catData <- data.frame(Name = c("Fluffy", "Max", "Coco"),
                      isFemale = c(TRUE, FALSE, TRUE),
                      weight.lbs = c(9.0, 11.5, 15.0),
                      Age = c(15, 5, 3))
print(catData)
```

	Name	isFemale	weight.lbs	Age
1	Fluffy	TRUE	9.0	15
2	Max	FALSE	11.5	5
3	Coco	TRUE	15.0	3

Accessing Data In Vectors, Matrices and Data Frames

```
# VECTORS

# extract the 3rd value in num.vec
val <- num.vec[3]
print(val)
```

[1] 7

```
# extract the second through fourth values from num.vec
vals <- num.vec[2:4]
print(vals)
```

```
[1] 4.5 7.0 12.2
```

```
# extract the first, third and fifth (non-contiguous) values from num.vec
idx <- c(1, 3, 5)
print(num.vec[idx])
```

```
[1] 10 7 3
```

```
# print the value in the 3rd row, 6th column of matrix.2D
print(matrix.2D[3,6])
```

```
[1] 8
```

```
# print the value in the 4th row, second column, third dimension of matrix.3D
print(matrix.3D[4,2,3])
```

```
[1] 16
```

```
# print Max's age from catData
print(catData[2,4])
```

```
[1] 5
```

```
# You can also
print(catData[2,"Age"])
```

```
[1] 5
```

Operators

Arithmetic

```
# Use operators with values only  
5 + 6
```

```
[1] 11
```

```
5 - 6
```

```
[1] -1
```

```
5 * 6
```

```
[1] 30
```

```
5 / 6
```

```
[1] 0.8333333
```

```
5 ^ 6
```

```
[1] 15625
```

```
5 ** 6
```

```
[1] 15625
```

```
# Define variables  
var1 = 2  
var2 = 8
```

```
# Use operators with vars only  
var1 + var2
```

```
[1] 10
```

```
var1 - var2
```

```
[1] -6
```

```
var1 * var2
```

```
[1] 16
```

```
var1 / var2
```

```
[1] 0.25
```

```
var1 ^ var2
```

```
[1] 256
```

```
var1 ** var2
```

```
[1] 256
```

```
# mix it up  
var1 + 5
```

```
[1] 7
```

```
var2 - 6
```

```
[1] 2
```

```
# you get the idea.
```

```
var1 = 10  
var2 = 3  
  
# Integer division  
var1 %/% 3
```

```
[1] 3
```

```
# Modulus  
var1 %% var2
```

```
[1] 1
```

```
# Both operators work with any combination of values and variables.
```

Relational

```
# Define some variables  
var1 = 5  
var2 = 7  
  
# Examine results of different relational expressions  
  
print(var1 < var2)
```

```
[1] TRUE
```

```
print(var1 >= var2)
```

```
[1] FALSE
```

```
print(var1 != var2)
```

```
[1] TRUE
```

```
# Define weights for Sparky & Bella  
wt.Sparky = 70  
wt.Bella = 70.01  
  
# Check if the two weights are equal  
wt.Sparky == wt.Bella
```

```
[1] FALSE
```

```

# We want to compare weights of three dogs under two scenarios.
# Define Sparky & Bella's weights again and add a third dog, Fido.
wt.Sparky = 70
wt.Bella = 70.01
wt.Fido = 70.1

# Scenario 1: Dogs weight the same if their integer weight in pounds is the same. [All dogs

precision.1 <- 1

# Check if weights are equal at the whole integer level
# Define equal.1 to check if Sparky's weight = Bella's weight
equal.1 <- abs(wt.Sparky-wt.Bella) < precision.1 # TRUE (e.g. weights are equal)

# Define equal.2 to check if Sparky's weight = Fido's weight

equal.2 <- abs(wt.Sparky-wt.Fido) < precision.1 # TRUE (e.g. weights are equal)

print(equal.1)

```

```
[1] TRUE
```

```
print(equal.2)
```

```
[1] TRUE
```

```

# Scenario 2: Dogs weight is the same if their integer weight in pounds is the same to the f

precision.2 <- .1

# Check if weights are equal to one decimal place
# Define equal.3 to check if Sparky's weight = Bella's weight
equal.3 <- abs(wt.Sparky-wt.Bella) < precision.2 # TRUE (e.g. weights are equal)

# Define equal.4 to check if Sparky's weight = Fido's weight
equal.4 <- abs(wt.Sparky-wt.Fido) < precision.2 # FALSE (e.g. weights are not equal)

print(equal.3)

```

```
[1] TRUE
```

```
print(equal.4)
```

```
[1] TRUE
```

```
# Let's print out the absolute difference between Sparky and Bella's weights
abs.dif <- abs(wt.Sparky-wt.Bella)
print(abs.dif) # prints out .01...Looks good!
```

```
[1] 0.01
```

```
# But....
# look at value of abs.dif in the Environment pane....?!?!
```

Logical

AND

```
# Using variables
# Define 3 variables and set them to boolean values
var.1 <- TRUE
var.2 <- FALSE
var.3 <- TRUE

# Define result.1 and result.2 each using two of the variables and the && operator
result.1 <- var.1 && var.2
result.2 <- var.1 && var.3

print(result.1)
```

```
[1] FALSE
```

```
print(result.2)
```

```
[1] TRUE
```

```
# Using expressions (relational)
# Define a variable and set it to an integer value
val.1 <- 5

# Define a variable, logical.1, and use it to determine if value is between two numbers

logical.1 <- (val.1 > 5) && (val.1 < 10)

print(logical.1)
```

[1] FALSE

OR

```
# Using variables
# Define 3 variables and set them to boolean values
var1 <- TRUE
var2 <- FALSE
var3 <- FALSE

# Define result.1 and result.2 each using two of the variables and the && operator
result.1 <- var.1 && var.2
result.2 <- var.1 && var.3

print(result.1)
```

[1] FALSE

```
print(result.2)
```

[1] TRUE

```
# Using expressions (relational)
# Define a variable and set it to an integer value
val.1 <- 5

# Define a variable, logical.1, and use it to determine if value is between two numbers
```

```
logical.1 <- (val.1 < 5) || (val.1 > 10)

print(logical.1)
```

[1] FALSE

NOT

```
# Define a boolean variable
var.1 <- FALSE

# Use the not operator
var.2 <- !(var.1)

print(var.2)
```

[1] TRUE
